Vulnerability CSV Report Generation

This skill provides guidance on generating structured CSV reports from vulnerability scan data—a com
mon format for security audits and compliance reporting.

Overview

CSV (Comma-Separated Values) is a widely-used format for security reports because it's:
- **Human-readable**: Can be opened in Excel, Google Sheets
- **Machine-parseable**: Easy to process programmatically
- **Universal**: Supported by all data analysis tools
- **Lightweight**: Smaller than JSON/XML formats

When to Use CSV Reports

Ideal Use Cases
- Compliance audits requiring tabular data
- Executive summaries for non-technical stakeholders
- Integration with ticketing systems (Jira, ServiceNow)
- Automated vulnerability tracking pipelines
- Data analysis in spreadsheet tools

Limitations
- No hierarchical data (flat structure only)
- Limited support for nested information
- No standard for binary data

Alternative formats: JSON (for APIs), PDF (for formal reports), HTML (for dashboards)

CSV Schema Design for Security Reports

Essential Fields

A well-designed vulnerability report CSV should include:

Field	Type	Description	Example
Package	String	Vulnerable package name	`express`
Version	String	Installed version	`4.17.1`
CVE_ID	String	Vulnerability identifier	`CVE-2022-24999`
Severity	Enum	Risk level	`CRITICAL`, `HIGH`
CVSS_Score	Float/String	Numeric severity score	`9.8` or `N/A`
Fixed_Version	String	Patched version	`4.18.0` or `N/A`
Title	String	Brief description	`XSS in Express.js`
Url	String	Reference link	`https://nvd.nist.gov/...`

Design Principles

1. **Use descriptive column names**: `Package` not `pkg`, `CVE_ID` not `id`
2. **Handle missing data**: Use `N/A` for unavailable fields, not empty strings
3. **Consistent data types**: Ensure all rows have same format
4. **Include metadata**: Consider adding scan date, target, tool version

Python CSV Generation with DictWriter

Why DictWriter?

Python's `csv.DictWriter` is ideal for structured reports:

- **Type-safe**: Column names defined upfront
- **Readable**: Use dictionary keys instead of indices
- **Maintainable**: Easy to add/remove columns
- **Automatic header generation**: No manual header writing

Basic Usage

```python
import csv

# Define schema
headers = ["Package", "Version", "CVE_ID", "Severity", "CVSS_Score",
           "Fixed_Version", "Title", "Url"]

# Prepare data
vulnerabilities = [
    {
        "Package": "minimist",
        "Version": "1.2.5",
        "CVE_ID": "CVE-2021-44906",
        "Severity": "CRITICAL",
        "CVSS_Score": 9.8,
        "Fixed_Version": "1.2.6",
        "Title": "Prototype Pollution",
        "Url": "https://avd.aquasec.com/nvd/cve-2021-44906"
    }
]

# Write CSV
with open('security_audit.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=headers)
    writer.writeheader()  # Write column names
    writer.writerows(vulnerabilities)  # Write all rows
```

Important parameters:
- `newline=''`: Prevents extra blank lines on Windows
- `encoding='utf-8'`: Handles special characters in descriptions

Severity-Based Filtering

Why Filter by Severity?

Security teams prioritize based on risk. Filtering ensures reports focus on critical issues:

Severity	Action Required	Typical SLA
CRITICAL	Immediate patch	24 hours
HIGH	Urgent patch	7 days
MEDIUM	Scheduled patch	30 days
LOW	Optional patch	90 days

Implementation

```python
def filter_high_severity(vulnerabilities, min_severity=['HIGH', 'CRITICAL']):
    """
    Filter vulnerabilities by severity level.
```

```
    Args:
        vulnerabilities: List of vulnerability dictionaries
        min_severity: List of severity levels to include

    Returns:
        Filtered list containing only specified severity levels
    """
    filtered = []
    for vuln in vulnerabilities:
        if vuln.get('Severity') in min_severity:
            filtered.append(vuln)
    return filtered

# Usage
all_vulns = [...]  # From scanner
critical_vulns = filter_high_severity(all_vulns, ['CRITICAL', 'HIGH'])
```

Field Mapping from JSON to CSV

Extracting Fields from Scanner Output

```python
import json

def parse_trivy_json_to_csv_records(json_file):
    """
    Parse Trivy JSON output and extract CSV-ready records.

    Returns list of dictionaries, one per vulnerability.
    """
    with open(json_file, 'r', encoding='utf-8') as f:
        data = json.load(f)

    records = []

    if 'Results' in data:
        for result in data['Results']:
            target = result.get('Target', 'Unknown')

            for vuln in result.get('Vulnerabilities', []):
                # Map JSON fields to CSV fields
                record = {
                    "Package": vuln.get('PkgName'),
                    "Version": vuln.get('InstalledVersion'),
                    "CVE_ID": vuln.get('VulnerabilityID'),
                    "Severity": vuln.get('Severity', 'UNKNOWN'),
                    "CVSS_Score": extract_cvss_score(vuln),
                    "Fixed_Version": vuln.get('FixedVersion', 'N/A'),
                    "Title": vuln.get('Title', 'No description'),
                    "Url": vuln.get('PrimaryURL', '')
                }
                records.append(record)

    return records

def extract_cvss_score(vuln):
```

```
    """Extract CVSS score (from cvss-score-extraction skill)."""
    cvss = vuln.get('CVSS', {})
    for source in ['nvd', 'ghsa', 'redhat']:
        if source in cvss:
            score = cvss[source].get('V3Score')
            if score is not None:
                return score
    return 'N/A'
```

Complete Vulnerability CSV Report Generator

```python
import json
import csv
import sys

def generate_vulnerability_csv_report(
    json_input,
    csv_output,
    severity_filter=['HIGH', 'CRITICAL']
):
    """
    Generate filtered CSV security report from Trivy JSON output.

    Args:
        json_input: Path to Trivy JSON report
        csv_output: Path for output CSV file
        severity_filter: List of severity levels to include
    """
    # Read JSON
    try:
        with open(json_input, 'r', encoding='utf-8') as f:
            data = json.load(f)
    except FileNotFoundError:
        print(f"[!] Error: Could not find {json_input}")
        sys.exit(1)

    # Extract and filter vulnerabilities
    vulnerabilities = []

    if 'Results' in data:
        for result in data['Results']:
            for vuln in result.get('Vulnerabilities', []):
                severity = vuln.get('Severity', 'UNKNOWN')

                # Apply severity filter
                if severity in severity_filter:
                    vulnerabilities.append({
                        "Package": vuln.get('PkgName'),
                        "Version": vuln.get('InstalledVersion'),
                        "CVE_ID": vuln.get('VulnerabilityID'),
                        "Severity": severity,
                        "CVSS_Score": get_cvss_score(vuln),
                        "Fixed_Version": vuln.get('FixedVersion', 'N/A'),
                        "Title": vuln.get('Title', 'No description'),
                        "Url": vuln.get('PrimaryURL', '')
                    })
```

```
    # Write CSV
    if vulnerabilities:
        headers = ["Package", "Version", "CVE_ID", "Severity",
                   "CVSS_Score", "Fixed_Version", "Title", "Url"]

        with open(csv_output, 'w', newline='', encoding='utf-8') as f:
            writer = csv.DictWriter(f, fieldnames=headers)
            writer.writeheader()
            writer.writerows(vulnerabilities)

        print(f"\n[SUCCESS] Found {len(vulnerabilities)} "
              f"{'/'.join(severity_filter)} vulnerabilities")
        print(f"[SUCCESS] Report saved to: {csv_output}")
    else:
        print(f"\n[SUCCESS] No {'/'.join(severity_filter)} vulnerabilities found")

def get_cvss_score(vuln_data):
    """Extract CVSS score with source priority."""
    cvss = vuln_data.get('CVSS', {})
    for source in ['nvd', 'ghsa', 'redhat']:
        if source in cvss:
            score = cvss[source].get('V3Score')
            if score is not None:
                return score
    return 'N/A'

# Usage
if __name__ == "__main__":
    generate_vulnerability_csv_report(
        json_input='trivy_report.json',
        csv_output='security_audit.csv',
        severity_filter=['CRITICAL', 'HIGH']
    )
```

Advanced Patterns

Pattern 1: Adding Metadata Row

```python
import csv
from datetime import datetime

# Add metadata as first row
metadata = {
    "Package": f"Scan Date: {datetime.now().isoformat()}",
    "Version": "Tool: Trivy v0.40.0",
    "CVE_ID": "Target: package-lock.json",
    "Severity": "", "CVSS_Score": "", "Fixed_Version": "",
    "Title": "", "Url": ""
}

with open('report.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=headers)
    writer.writeheader()
    writer.writerow(metadata)  # Metadata row
    writer.writerow({})  # Blank separator
```

```
    writer.writerows(vulnerabilities)  # Actual data
```

Pattern 2: Multi-Target Reports

```python
def generate_multi_target_report(json_input, csv_output):
    """Include target/file name in each row."""
    with open(json_input, 'r') as f:
        data = json.load(f)

    vulnerabilities = []

    for result in data.get('Results', []):
        target = result.get('Target', 'Unknown')

        for vuln in result.get('Vulnerabilities', []):
            record = {
                "Target": target,  # Add target column
                "Package": vuln.get('PkgName'),
                # ... other fields
            }
            vulnerabilities.append(record)

    headers = ["Target", "Package", "Version", ...]  # Target first
    # Write CSV as before
```

Pattern 3: Summary Statistics

```python
def print_report_summary(vulnerabilities):
    """Print summary before writing CSV."""
    from collections import Counter

    severity_counts = Counter(v['Severity'] for v in vulnerabilities)

    print("\nVulnerability Summary:")
    print(f"  CRITICAL: {severity_counts.get('CRITICAL', 0)}")
    print(f"  HIGH:     {severity_counts.get('HIGH', 0)}")
    print(f"  Total:    {len(vulnerabilities)}")
```

Error Handling

Handling Missing or Malformed Data

```python
def safe_get_field(vuln, field, default='N/A'):
    """Safely extract field with default fallback."""
    value = vuln.get(field, default)
    # Ensure value is not None
    return value if value is not None else default

# Usage in field mapping
record = {
    "Package": safe_get_field(vuln, 'PkgName', 'Unknown'),
    "Fixed_Version": safe_get_field(vuln, 'FixedVersion', 'N/A'),
```

```
    # ...
}
```

Best Practices

1. **Always write headers**: Makes CSV self-documenting
2. **Use UTF-8 encoding**: Handles international characters
3. **Set newline=''**: Prevents blank lines on Windows
4. **Validate data**: Check for None/null values before writing
5. **Add timestamp**: Include scan date for tracking
6. **Document schema**: Maintain a data dictionary
7. **Test with edge cases**: Empty results, missing fields

Dependencies

Python Modules
- `csv` (standard library)
- `json` (standard library)

Input Format
- Requires structured vulnerability data (typically JSON from scanners)

References

- [Python CSV Documentation](https://docs.python.org/3/library/csv.html)
- [RFC 4180 - CSV Format Specification](https://tools.ietf.org/html/rfc4180)
- [NIST Vulnerability Database](https://nvd.nist.gov/)